



Cairo University

Cairo University, Faculty of Engineering
Electronics and Electrical Communication

Department (EECE)



Communication project

Source Coding (Part 2)

Submitted to: Dr. Mohammed Nafie

TA: Eng. Mohammed Khaled

Team T5

Name	Sec	Id
Dalia Mohamed Soliman Mohamed	2	9220285
Roa Atef Mohamed Abdelmaged	2	9220291
Sarah Abdelatty Ibrahim Abdelatty	2	9220349
Sama Saeed Mohamed Mohamed	2	9220371
Sohaila Ahmed Mohamed Abdelmaged	2	9220377

Contents

Question 3:.....	4
❖ Matlab code	4
❖ Output Figure	6
❖ Comments on Figure	6
Question 4:.....	7
❖ Matlab code	7
❖ Output Figure	9
❖ Comments on Figure	9
Question 5:.....	11
❖ Matlab code	11
❖ Output Figure	14
❖ Comments on Figure	14
Question 6:.....	16
❖ Matlab code	16
❖ Output Figure	20
❖ Comments on Figure	21
Question 7:.....	22
❖ Matlab code	22
❖ Output Figure	26
❖ Comments on Figure	26
Question 8:.....	28
❖ Matlab code	28
❖ Output Figure	32
❖ Comments on Figure	32
Question 9:.....	33
❖ Matlab code	33
❖ Output Figure	36
❖ Comments on Figure	36
Appendix: Full Matlab Code.....	37

Table of Figures

Figure 1	: BPSK (BER vs Eb/No).....	5
Figure 2	:Uncoded BPSK vs Repetition-3 Coding.....	8
Figure 3	: Uncoded BPSK vs Repetition-3 Coding using Soft decision decoding.....	13
Figure 4	: BER of BPSK uncoded vs Hamming (7,4) Coding.....	18
Figure 5	: BER of BPSK uncoded vs Hamming (15,11) Coding.....	24
Figure 6	: BER of uncoded BPSK vs QPSK with Hamming (15,11) Coding.....	25
Figure 7	: Figure 6: BER for QPSK uncoded vs 16-QAM coded.....	30
Figure 8	: Convolution Encoding.....	34
Figure 9	: Output bit stream.....	34

Question 3:

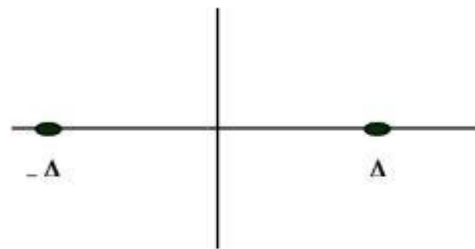
Assume you want to transmit 110000 information bits.

Use BPSK with no coding and plot a graph showing the probability of error versus

E_b/N_0 for $E_b/N_0 = -3$ to 10 dB, where $E_b = A^2$

$$\sigma = \sqrt{(E_b/2) \cdot 10^{-(E_b/N_0/10)}};$$

where σ is noise power per dimension, E_b is energy per bit, and E_b/N_0 is E_b/N_0 .



BPSK Constellation

❖ Matlab code

```
% Number of bits
N = 110000;

% Generate random bits (0/1)
bits = randi([0 1], N, 1);

% BPSK mapping: 0 -> -A , 1 -> +A
A = 1; % amplitude
Eb = A^2; % energy per bit
symbols = 2*bits - 1; % gives -1 and +1

% Eb/No range EbNo_dB = -
3:1:10; ber =
zeros(size(EbNo_dB)); for
i = 1:length(EbNo_dB)
```

```

% Convert Eb/No from dB to linear
EbNo = 10^(EbNo_dB(i)/10);

% Noise variance per dimension (given in question)
sigma = sqrt((Eb/2) / EbNo);

% AWGN noise
noise = sigma * randn(N,1);

% Received signal r
= symbols + noise;

% Detection (threshold at 0)
detected_bits = r > 0;

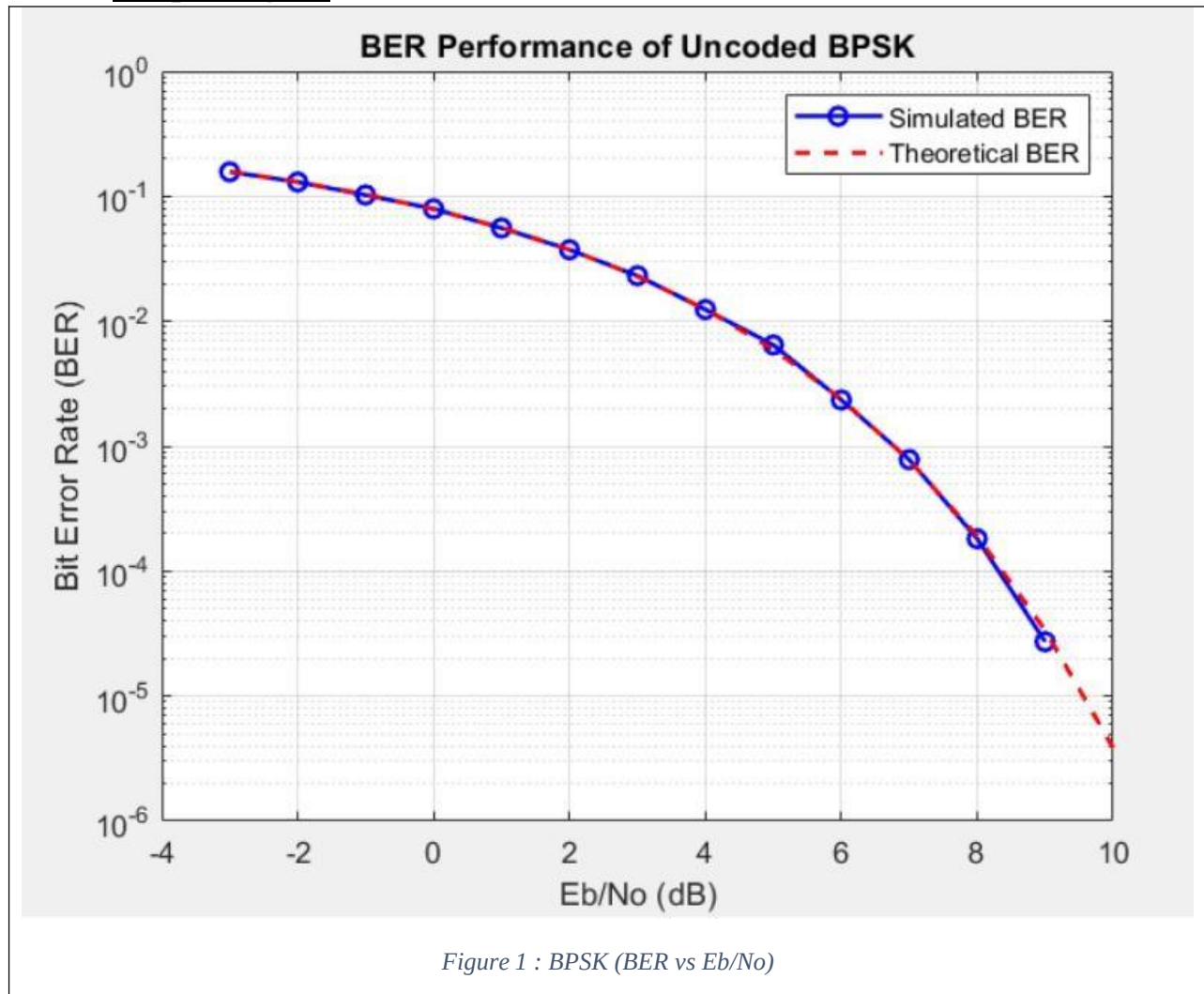
% Compute BER
ber(i) = sum(bits ~= detected_bits) / N;
end

% Theoretical BER for BPSK
EbNo_linear = 10.^(EbNo_dB/10);
ber_theoretical = qfunc(sqrt(2*EbNo_linear));

% Plot results
figure;
semilogy(EbNo_dB, ber, 'bo-', 'LineWidth', 1.5); hold on;
semilogy(EbNo_dB, ber_theoretical, 'r--', 'LineWidth', 1.5);
grid on;
xlabel('Eb/No (dB)'); ylabel('Bit Error Rate (BER)'); title('BER Performance of Uncoded BPSK'); legend('Simulated BER','Theoretical BER');

```

❖ Output Figure



❖ Comments on Figure

As E_b/N_0 increases, the BER decreases because the signal becomes stronger relative to the noise, leading to fewer detection errors.

At low E_b/N_0 values, noise dominates the signal, resulting in a high BER.

Question 4:

Repeat 1, but use BPSK with repetition 3 coding, and use hard decision decoding:

(a) With same energy per transmitted bit.

(b) With same energy per information bit.

Plot each of the curves of (a) and (b) with the curve of 1 on the same graph.

❖ Matlab code

```
%% Parameters
Rep = 3; % Repetition factor
Number_of_Information_Bits = 110000;
Information_bits = randi([0,1], 1,
Number_of_Information_Bits);
EbNo_dB = -4:1:10; % Eb/No range N_EbNo =
length(EbNo_dB);

%% --- BPSK with No Code
A = 1;
Eb = A^2;
Mapped_Data = (2*Information_bits - 1) * A;
BER_No_Code = zeros(1, N_EbNo);

for idx = 1:N_EbNo ebno
    = EbNo_dB(idx);
    Sigma = sqrt((Eb/2) / 10^(ebno/10));
    Noise = randn(1, Number_of_Information_Bits);
    Rx = Mapped_Data + Sigma*Noise;

    DeMapped_Signal = Rx > 0; % 0/1 decision
    BER_No_Code(idx) = sum(DeMapped_Signal ~=
Information_bits)/Number_of_Information_Bits;
end

%% --- Repetition-3: Same Energy per Transmitted Bit
Information_bits_rep = repelem(Information_bits, Rep);
A = 1; % same energy per transmitted bit
Eb = A^2;
Mapped_Data_rep = (2*Information_bits_rep - 1) * A;
BER_HARD_1 = zeros(1, N_EbNo);
```

```

for idx = 1:N_EbNo ebno
    = EbNo_dB(idx);
    Sigma = sqrt((Eb/2)/10^(ebno/10));
    Noise = randn(1, length(Information_bits_rep));
    Rx = Mapped_Data_rep + Sigma*Noise;

    DeMapped_Signal = Rx > 0; % 0/1 decision
    Restored_Signal = zeros(1, Number_of_Information_Bits);

    for k = 1:Number_of_Information_Bits group =
        DeMapped_Signal((k-1)*Rep + 1 : k*Rep);
        Restored_Signal(k) = sum(group) >= 2; % majority vote
    end

    BER_HARD_1(idx) = sum(Restored_Signal ~=
Information_bits)/Number_of_Information_Bits;
end

%% --- Repetition-3: Same Energy per Information Bit A =
1/sqrt(Rep); % scale amplitude to keep energy per info bit
same
Eb = 1;
Mapped_Data_rep = (2*Information_bits_rep - 1) * A;
BER_HARD_2 = zeros(1, N_EbNo);

for idx = 1:N_EbNo ebno
    = EbNo_dB(idx);
    Sigma = sqrt((Eb/2)/10^(ebno/10));
    Noise = randn(1, length(Information_bits_rep));
    Rx = Mapped_Data_rep + Sigma*Noise;

    DeMapped_Signal = Rx > 0; % 0/1 decision
    Restored_Signal = zeros(1, Number_of_Information_Bits);

    for k = 1:Number_of_Information_Bits group =
        DeMapped_Signal((k-1)*Rep + 1 : k*Rep);
        Restored_Signal(k) = sum(group) >= 2; % majority vote
    end

    BER_HARD_2(idx) = sum(Restored_Signal ~=
Information_bits)/Number_of_Information_Bits;
end

```



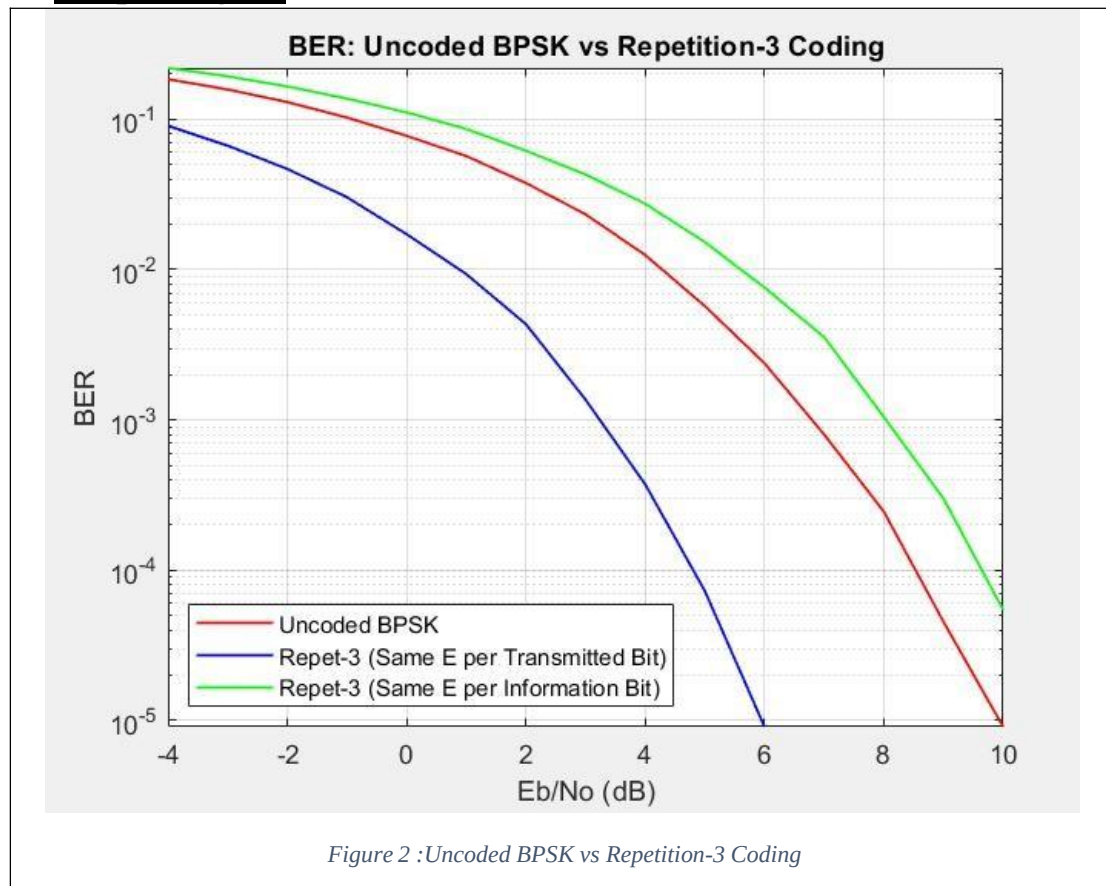
```

%% --- Plot BER
figure;
semilogy(EbNo_dB, BER_No_Code, 'r', 'LineWidth', 1); hold
on;
semilogy(EbNo_dB, BER_HARD_1, 'b', 'LineWidth', 1);
semilogy(EbNo_dB, BER_HARD_2, 'g', 'LineWidth', 1);

grid on; axis tight;
xlabel('Eb/No (dB)');
ylabel('BER');
title('BER: Uncoded BPSK vs Repetition-3 Coding');
legend('Uncoded BPSK', ...
       'Repet-3 (Same E per Transmitted Bit)', ...
       'Repet-3 (Same E per Information Bit)', ...
       'Location', 'southwest');

```

❖ Output Figure



❖ Comments on Figure

1. **Repetition-3 with same energy per information bit (highest BER):**
 - Repeating each bit reduces the energy per transmitted bit, so each copy is weaker and more affected by noise, leading to more errors.
2. **Repetition-3 with same energy per transmitted bit (lowest BER):**

- Here, each transmitted bit has the same energy as the uncoded bit, so repeating it increases the total energy per information bit.

Overall: The plot shows that **repetition coding improves BER only if the energy per transmitted bit is maintained**, while keeping total energy per information bit the same may actually worsen performance.

Question & Answer :

If you are designing a communication system, can you think of a scenario where you will use such an error correction coding scheme as in (a) or (b)?

In designing a communication system, the choice of error correction coding depends on the system's energy and reliability requirements.

- **Repetition-3 coding with the same energy per transmitted bit** (case a) is useful in scenarios where reliability is critical, and the transmitter can afford higher total energy per information bit.
- **Repetition-3 coding with the same energy per information bit** (case b) is suitable for energy-constrained systems. Here, the total energy per information bit is fixed, so each transmitted copy has lower energy. While this provides only limited error protection, it allows simple coding without increasing power consumption.

In summary, case (a) prioritizes reliability at the cost of energy, whereas case (b) prioritizes energy efficiency while offering modest error correction.

Question 5:

Repeat 2, but use soft decision decoding. Again, think of a scenario where you will use such an error correction coding scheme.

❖ Matlab code

```
%% Parameters

Rep = 3; % Repetition factor

Number_of_Information_Bits = 110000;

Information_bits = randi([0,1], 1,
Number_of_Information_Bits);

EbNo_dB = -4:1:10; % Eb/No range

N_EbNo = length(EbNo_dB);

%% --- BPSK with No Code

A = 1;

Eb = A^2;

Mapped_Data = (2*Information_bits - 1) *
A; BER_No_Code = zeros(1, N_EbNo); for idx
= 1:N_EbNo ebno = EbNo_dB(idx);

Sigma = sqrt((Eb/2) / 10^(ebno/10));

Noise = randn(1, Number_of_Information_Bits);

Rx = Mapped_Data + Sigma*Noise;

DeMapped_Signal = Rx > 0; % 0/1 decision

BER_No_Code(idx) = sum(DeMapped_Signal ~=
Information_bits)/Number_of_Information_Bits;

end

%% --- Repetition-3: Same Energy per Transmitted Bit
```

```

Information_bits_rep = repelem(Information_bits, Rep);
A = 1; % same energy per transmitted bit
Eb = A^2;
Mapped_Data_rep = (2*Information_bits_rep - 1) * A;
BER_SOFT_1 = zeros(1, N_EbNo); for idx = 1:N_EbNo
ebno = EbNo_dB(idx);
Sigma = sqrt((Eb/2)/10^(ebno/10));
Noise = randn(1,
length(Information_bits_rep)); Rx =
Mapped_Data_rep + Sigma*Noise; groups_a =
reshape(Rx, Rep, []);

sum_soft_a = sum(groups_a, 1);
det_soft_a = sum_soft_a > 0;

BER_SOFT_1(idx) = sum(det_soft_a ~=
Information_bits)/Number_of_Information_Bits;
end

%% --- Repetition-3: Same Energy per Information Bit
A = 1/sqrt(Rep); % scale amplitude to keep energy per
info bit same
Eb = 1;

Mapped_Data_rep = (2*Information_bits_rep - 1) * A;
BER_SOFT_2 = zeros(1, N_EbNo); for idx = 1:N_EbNo
ebno = EbNo_dB(idx);

```

```

Sigma = sqrt((Eb/2)/10^(ebno/10));
Noise = randn(1,
length(Information_bits_rep)); Rx =
Mapped_Data_rep + Sigma*Noise; groups_b =
reshape(Rx, Rep, []);

sum_soft_b = sum(groups_b, 1);
det_soft_b = sum_soft_b > 0;

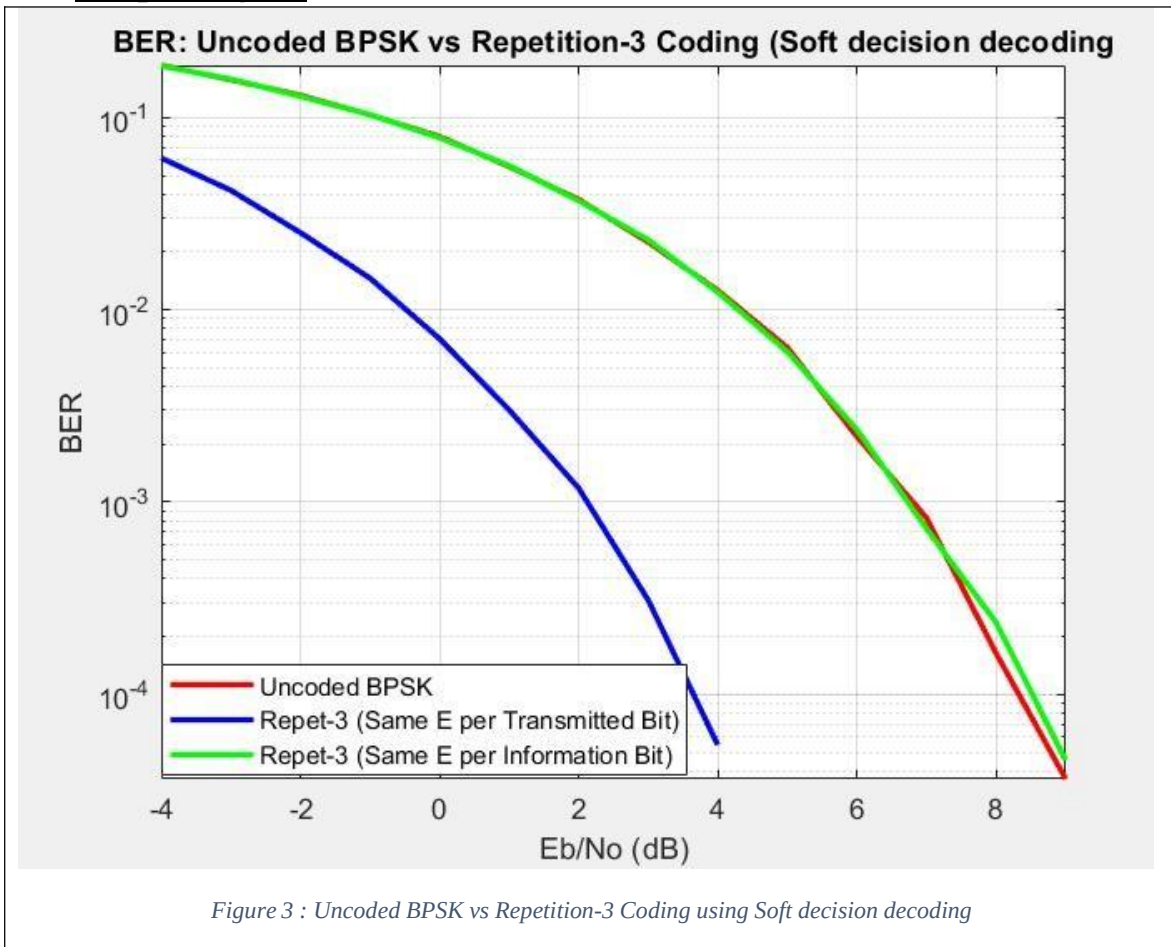
BER_SOFT_2(idx) = sum(det_soft_b ~=
Information_bits)/Number_of_Information_Bits;
end

%% --- Plot BER

figure;
semilogy(EbNo_dB, BER_No_Code, 'r', 'LineWidth', 2);
hold on; semilogy(EbNo_dB, BER_SOFT_1, 'b',
'LineWidth', 2); semilogy(EbNo_dB, BER_SOFT_2, 'g',
'LineWidth', 2); grid on; axis tight; xlabel('Eb/No
(dB)'); ylabel('BER'); title('BER: Uncoded BPSK vs
Repetition-3 Coding (Soft decision decoding)');
legend('Uncoded BPSK', ...
'Repetition-3 (Same E per Transmitted Bit)', ...
'Repetition-3 (Same E per Information Bit)', ...
'Location', 'southwest');

```

❖ Output Figure



❖ Comments on Figure

The figure compares uncoded BPSK with repetition-3 coding, using uncoded BPSK as a baseline. With the same energy per transmitted bit, repetition-3 coding significantly improves BER due to redundancy. However, with the same energy per information bit, repetition-3 coding shows nearly the same BER as uncoded BPSK.

❖ Question Answer :

1) Scenario_1: Sufficient Energy Available

- **Approach:** Use repetition-3 coding **with the same power per transmitted bit**.
- **Effect:** Each bit is transmitted three times without reducing energy per bit.
- **Result:** Strong error correction → **best BER performance**
- **When to use:** Systems where **BER is critical**, and energy availability is not a limiting factor.

2) Scenario: Energy Efficiency Needed

- **Approach:** Use repetition-3 coding **with the same power per information bit**.
- **Effect:** To keep total energy per information bit constant, the energy per transmitted bit is reduced by a factor of 3.
- **Result:**

- Less improvement in BER compared to the first case
- **Energy is conserved** → more efficient for battery-operated or power-limited systems
- **When to use:** Systems where **energy consumption is critical**, and some degradation in BER is acceptable.

Question 6:

Repeat 2 but use BPSK with a (7,4) Hamming code. You can use Matlab “encode” and “decode” functions.

(c) What is the minimum distance of this Hamming code?

(d) Would you recommend such a code if the purpose is to decrease the bit error rate using the same energy, but disregarding transmit time?

❖ Matlab code

```
%% ===== Parameters =====  
bitsNumber = 110000; % Number of information bits  
Eb = 1; % Energy per information bit A =  
1; % BPSK signal amplitude ebno_dB = -  
2:1:10; % Eb/N0 range in dB n = 7; %  
Hamming codeword length (7 bits) k = 4;  
% Hamming message length (4 bits) rate =  
k/n; % Code rate of Hamming (7,4)  
  
%% ===== Generate random bits  
=====
```

```
Bits = randi([0 1], 1, bitsNumber); % Generate random  
binary information bits
```

```
%% ===== BPSK Without Coding  
=====
```

```
BPSKSymbols = A * (2*Bits - 1); % Map 0->-1, 1->+1 for  
BPSK
```

```
BER_uncoded = zeros(1,length(ebno_dB));
```



```

for idx = 1:length(ebno_dB) EbN0 = ebno_dB(idx);
sigma = sqrt((Eb/2)/10^(EbN0/10)); % Noise
standard deviation for AWGN
Noise = sigma * randn(1,bitsNumber); % Generate AWGN
noise
Received = BPSKSymbols + Noise; % Received signal
after noise
ReceivedBits = Received > 0; % Hard decision
ErrorBits = sum(Bits ~= double(ReceivedBits)); % Count
bit errors
BER_uncoded(idx) = ErrorBits / bitsNumber;
end

%% ===== BPSK with Hamming (7,4) Same
Energy per Transmitted Bit =====
HammCodeBits = encode(Bits, n, k, 'hamming/binary'); %
Encode using Hamming (7,4)
A = 1; % Keep amplitude same per transmitted bit
BPSKSymbols_case_a = A * (2*HammCodeBits - 1); % BPSK
mapping for coded bits
BER_case_a =
zeros(1,length(ebno_dB)); for idx =
1:length(ebno_dB) EbN0 =
ebno_dB(idx);
sigma = sqrt((Eb/2)/10^(EbN0/10));
Noise = sigma * randn(1,length(HammCodeBits)); %
Generate noise
Received = BPSKSymbols_case_a + Noise; % Received
signal

```

```
ReceivedBits = Received > 0; % Hard decision
```

```

DecodedBits = decode(ReceivedBits, n, k,
'hamming/binary'); % Decode
ErrorBits = sum(Bits ~=
double(DecodedBits)); BER_case_a(idx) =
ErrorBits / bitsNumber; end

%% ===== BPSK with Hamming (7,4)
Same Energy per Information Bit
=====

A = sqrt(rate); % Scale amplitude to keep same
energy per info bit
BPSKSymbols_case_b = A * (2*HammCodeBits - 1); %
BPSK mapping
BER_case_b = zeros(1,length(ebno_dB)); %
Preallocate BER array

for idx = 1:length(ebno_dB)
EbN0 = ebno_dB(idx);
sigma = sqrt((Eb/2)/10^(EbN0/10));
Noise = sigma * randn(1,length(HammCodeBits));
Received = BPSKSymbols_case_b + Noise; % Received
signal
ReceivedBits = Received > 0; % Hard decision
DecodedBits = decode(ReceivedBits, n, k,
'hamming/binary'); % Decode
ErrorBits = sum(Bits ~=
double(DecodedBits)); BER_case_b(idx) =
ErrorBits / bitsNumber; end

%% ===== Plot BER Comparison

```

```

=====
figure; semilogy(ebno_dB, BER_uncoded,'k-
','LineWidth',2); hold on; semilogy(ebno_dB,
BER_case_a,'r-','LineWidth',2); semilogy(ebno_dB,
BER_case_b,'b-','LineWidth',2); grid on;
xlabel('E_b/N_0 (dB)'); ylabel('BER');
title('BER Comparison of BPSK uncoded and with
Hamming (7,4) Coding'); legend('BPSK
Uncoded',...
'BPSK Hamming (Same Energy per Transmission
Bit)',...
'BPSK Hamming (Same Energy per Info Bit)');

```

❖ Output Figure

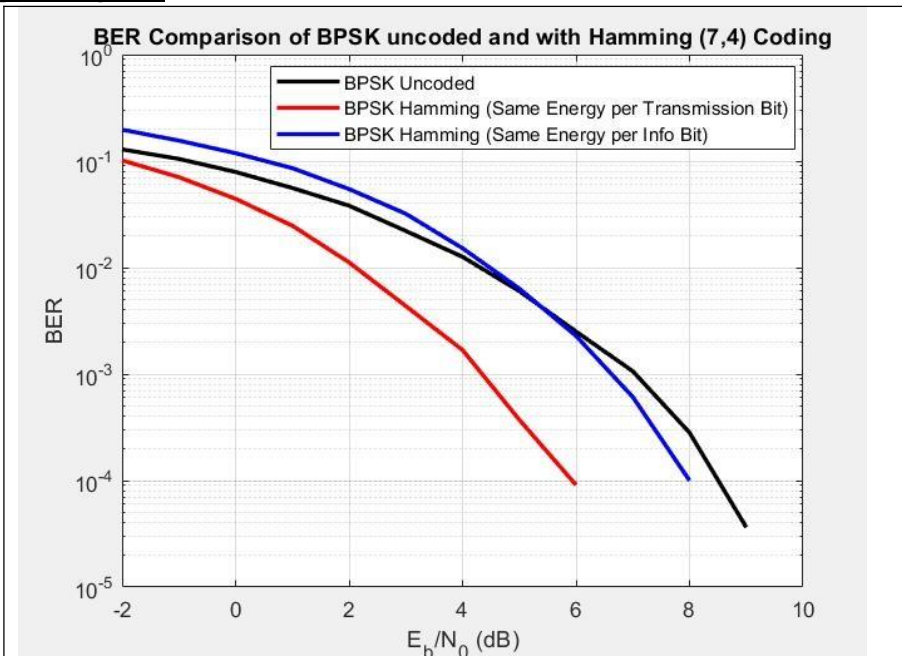


Figure 4 : BER of BPSK uncoded vs Hamming (7,4) Coding

❖ Comments on Figure

The performance of Hamming (7,4) depends on energy allocation.

- With the same energy per transmitted bit, redundancy increases the energy per information bit, significantly lowering BER.
- With the same energy per information bit, energy is spread over more bits, so error correction is limited at low E_b/N_0 .

❖ **Answer (c) :**

Minimum distance = 3. This means it can **detect up to 2 errors** and **correct 1 error** per codeword.

❖ **Answer (d) :**

If the goal is to decrease the bit error rate (BER) while keeping the same energy per transmitted bit and ignoring transmit time, then yes, the (7,4) Hamming code is recommended because Adding redundancy increases the effective energy per information bit and allows error correction, reducing the BER.

Question 7:

Repeat 2 but use BPSK with a (15,11) Hamming code.

(e) Would you recommend such a code if the purpose is to decrease the bit error rate using the same energy, but disregarding transmit time? (f) If you want to keep transmission time equal to or less than the transmission time of 1, what would you propose?

(g) Plot a curve for the performance of your proposal in (b). Comment.

❖ Matlab code

```
%% ===== Parameters =====
bitsNumber = 110000; % Number of information bits to
simulate
Eb = 1; % Energy per information bit A = 1; % BPSK
signal amplitude ebno_dB = -3:1:10; % Eb/N0 range in
dB for simulation n = 15; % Hamming codeword length
(15 bits) k = 11; % Hamming message length (11 bits)
rate = k/n; % Hamming code rate (11/15)

%% ===== Generate random bits
=====
Bits = randi([0 1], 1, bitsNumber); % Generate random
binary information bits

%% ===== BPSK Without Coding
=====
```

```

BPSKSymbols = A * (2*Bits - 1); % Map 0->-1, 1->+1 for BPSK
BER_uncoded = zeros(1,length(ebno_dB));

for idx = 1:length(ebno_dB) EbN0 = ebno_dB(idx);
sigma = sqrt((Eb/2)/10^(EbN0/10)); % Noise
standard deviation for AWGN
Noise = sigma * randn(1,bitsNumber); % Generate AWGN noise
Received = BPSKSymbols + Noise; % Received signal after
noise
ReceivedBits = Received > 0; % Hard decision demodulation
ErrorBits = sum(Bits ~= double(ReceivedBits)); % Count bit
errors
BER_uncoded(idx) = ErrorBits / bitsNumber; % Compute BER
end

%% ===== BPSK with Hamming (15,11) Same
Energy per Transmitted Bit =====
HammCodeBits = encode(Bits, n, k, 'hamming/binary'); %
Encode using Hamming (15,11)
A_tx = 1; % Amplitude unchanged: same energy per
transmitted bit
BPSKSymbols_Hamm_tx = A_tx * (2*HammCodeBits - 1); % BPSK
mapping for coded bits
BER_Hamm_1 = zeros(1,length(ebno_dB));

for idx = 1:length(ebno_dB)
EbN0 = ebno_dB(idx);

```

```

sigma = sqrt((Eb/2)/10^(EbN0/10)); % Noise standard
deviation
Noise = sigma * randn(1,length(HammCodeBits)); % AWGN noise
Received = BPSKSymbols_Hamm_tx + Noise; % Received signal
ReceivedBits = Received > 0; % Hard decision
DecodedBits = decode(ReceivedBits, n, k,
'hamming/binary'); % Decode Hamming
ErrorBits = sum(Bits ~= double(DecodedBits)); % Count
errors
BER_Hamm_1(idx) = ErrorBits / bitsNumber; % Compute BER
end

%% ===== BPSK with Hamming (15,11) Same
Energy per Information Bit =====
A_ib = sqrt(rate); % Scale amplitude to keep same energy
per information bit
BPSKSymbols_Hamm_ib = A_ib * (2*HammCodeBits - 1); % BPSK
mapping
BER_Hamm_2 = zeros(1,length(ebno_dB)); % Preallocate BER
array

for idx = 1:length(ebno_dB) EbN0 = ebno_dB(idx);
sigma = sqrt((Eb/2)/10^(EbN0/10)); % Noise
standard deviation
Noise = sigma * randn(1,length(HammCodeBits));
Received = BPSKSymbols_Hamm_ib + Noise; % Received signal
ReceivedBits = Received > 0; % Hard decision

```



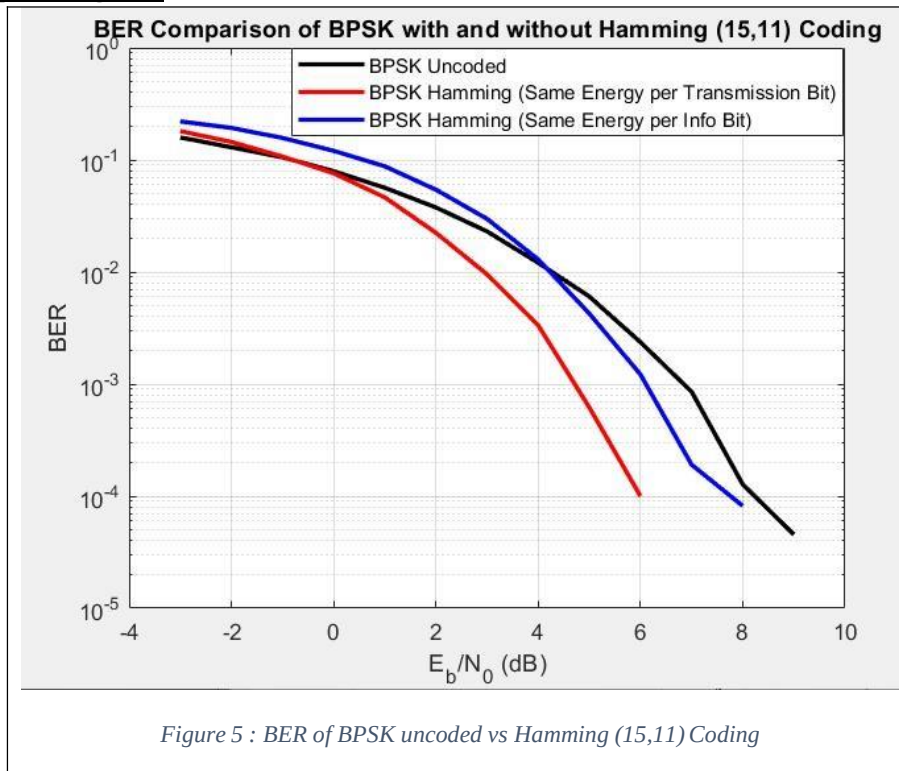
```

DecodedBits = decode(ReceivedBits, n, k,
'hamming/binary'); % Decode Hamming
ErrorBits = sum(Bits ~= double(DecodedBits)); % Count
errors
BER_Hamm_2(idx) = ErrorBits / bitsNumber; % Compute BER
end

%% ===== Plot BER Comparison
===== figure; semilogy(ebno_dB,
BER_uncoded,'k-','LineWidth',2); hold on;
semilogy(ebno_dB, BER_Hamm_1,'r-','LineWidth',2);
semilogy(ebno_dB, BER_Hamm_2,'b-','LineWidth',2); grid on;
xlabel('E_b/N_0 (dB)'); ylabel('BER'); title('BER
Comparison of BPSK with and without Hamming
(15,11) Coding');
legend('BPSK Uncoded',...
'BPSK Hamming (Same Energy per Transmission Bit)',...
'BPSK Hamming (Same Energy per Info Bit)');

```

❖ Output Figure



❖ Comments on Figure

The plot shows that Hamming (15,11) significantly reduces BER compared to uncoded BPSK. The gain is highest when keeping the same energy per transmitted bit.

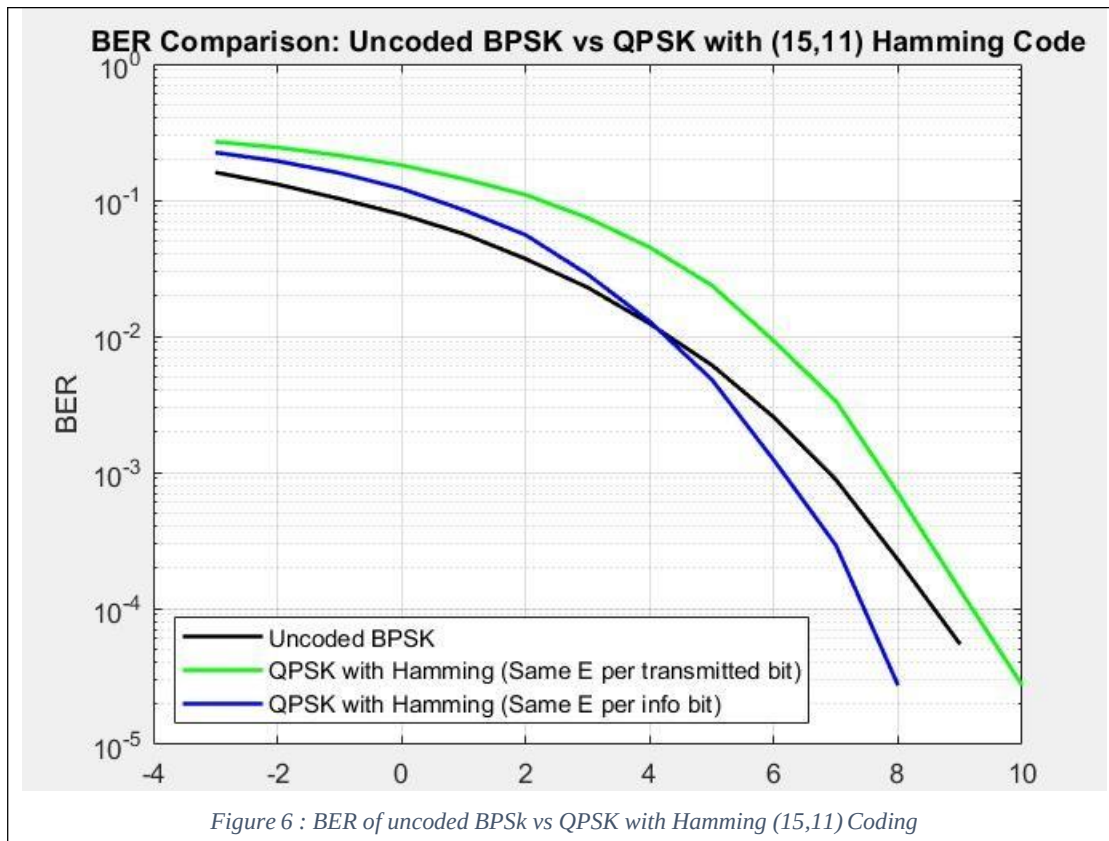
❖ Answer (e)

Yes, it is recommended Because the Hamming (15,11) code provides coding gain and improves BER when energy is fixed and time is not a constraint.

❖ Answer (f):

Transmission time is proportional to the number of transmitted bits, so coding with BPSK increases transmission time. To keep the transmission time equal to or less than uncoded BPSK, spectral efficiency must be increased. This can be achieved by using Hamming (15,11) coded QPSK, which maintains time while providing error correction.

❖ Answer (g):



The figure shows that Hamming (15,11) coded QPSK achieves a lower BER than uncoded BPSK at moderate and high E_b/N_0 due to coding gain.

Question 8:

Now, assume you want to transmit 26200000 bit. Compare, on the same curve, at $E_b/N_0 = 5$ to 15 dB, between (h) Using QPSK with no coding.

(i) Using 16 QAM but utilizing a (255,131) BCH code. Use Matlab “encode” and “decode” functions. Also, you can use the following functions for 16 QAM modulation and demodulation.

Notes:

(i) Use $E_b = 2.5$.

(ii) Simulation time: To speed up simulation time, at $E_b/N_0 = 5$ to 8 you can only do 2620000 bits. You can, for example, do a loop 100 times for 26200 bits each, or whatever is faster on your computer. For $E_b/N_0 = 9$ you will have to do 26200000 bits at least. For $E_b/N_0 = 10$ to 15 you can assume linear interpolation between 8 and 15. Plot the curve from 5 to 15 dB.

❖ Matlab code

```
%% ===== Clear workspace =====
clear; close all; clc;

%% ===== PARAMETERS =====
Eb = 2.5;
EbNo_dB = 5:15; % Eb/No range in dB
Nbits_total = 26200000;

% BCH code parameters
(255,131) n_bch = 255; k_bch = 131;

%% ===== BCH Encoder/Decoder
===== bchEnc = comm.BCHEncoder(n_bch, k_bch);
bchDec = comm.BCHDecoder(n_bch, k_bch);

%% ===== BER Arrays
===== ber_qpsk =
zeros(size(EbNo_dB)); ber_16qam =
zeros(size(EbNo_dB));

%% ===== QPSK SIMULATION
===== for idx = 1:length(EbNo_dB) EbNo =
10^(EbNo_dB(idx)/10);
```

```

% Adaptive number of bits
if EbNo_dB(idx) <= 8
Nbits = 2620000; else
Nbits = Nbits_total;
end
% Generate random bits
txBits = randi([0 1], 1, Nbits);
% QPSK mapping: 00->1+j, 01->-1+j, 11->-1-j, 10->1-j
txSym = 1-2*txBits(1:2:end) + 1j*(1-2*txBits(2:2:end));
% Add AWGN
N0 = 2*Eb/(2*EbNo); % QPSK has 2 bits/symbol
noise =
sqrt(N0/2)*(randn(size(txSym))+1j*randn(size(txSym)));
rxSym = txSym + noise; % QPSK demodulation
rxBits = zeros(1,Nbits); rxBits(1:2:end) = real(rxSym)<0;
rxBits(2:2:end) = imag(rxSym)<0;
% BER computation
ber_qpsk(idx) = sum(rxBits~=txBits)/Nbits;
end

%% ===== 16-QAM + BCH SIMULATION
===== for idx =
1:length(EbNo_dB) EbNo =
10^(EbNo_dB(idx)/10); %
Adaptive number of bits if
EbNo_dB(idx) <= 8 Nbits =
2620000; else
Nbits = Nbits_total;
end
% Generate random bits txBits = randi([0
1], 1, Nbits); % BCH encoding block by
block nBlocks =
ceil(length(txBits)/k_bch); encBits =
zeros(1, nBlocks*n_bch); for blk =
1:nBlocks startIdx = (blk-1)*k_bch + 1;
endIdx = min(blk*k_bch, length(txBits));
dataBlock = txBits(startIdx:endIdx);
% Pad last block if needed

```

```

if length(dataBlock) < k_bch
dataBlock = [dataBlock zeros(1,k_bch-length(dataBlock))];
end
% Convert to column vector
dataBlockCol = dataBlock(:);
% Encode
encBlockCol = step(bchEnc, dataBlockCol);
% Store in row vector
encBits((blk-1)*n_bch + 1 : blk*n_bch) = encBlockCol(:)';
end
% 16-QAM modulation using provided function
txSym = mod16(encBits);
% Add AWGN
m = 4; % 16-QAM: 4 bits/symbol
N0 = 4*Eb/(2*EbNo); % 16-QAM has 4 bits/symbol
noise =
sqrt(N0/2)*(randn(size(txSym))+1j*randn(size(txSym)));
rxSym = txSym + noise;
% 16-QAM demodulation using provided
function rxBits = demod16(rxSym); % BCH
decoding block by block decBits =
zeros(1,length(txBits)); for blk = 1:nBlocks
startIdx = (blk-1)*n_bch + 1; endIdx =
blk*n_bch;
rxBlock = rxBits(startIdx:endIdx);
% Convert to column vector
rxBlockCol = rxBlock(:);
% Decode
decBlockCol = step(bchDec, rxBlockCol);
% Trim padding for last block
if blk==nBlocks && length(decBlockCol) > length(txBits) -
(nBlocks-1)*k_bch
decBlockCol = decBlockCol(1:length(txBits) - (nBlocks-
1)*k_bch);
end
decBits((blk-1)*k_bch + 1 : min(blk*k_bch,length(txBits)))
= decBlockCol(:)';
end
% BER computation
ber_16qam(idx) = sum(decBits~=txBits)/length(txBits);
end

```

```

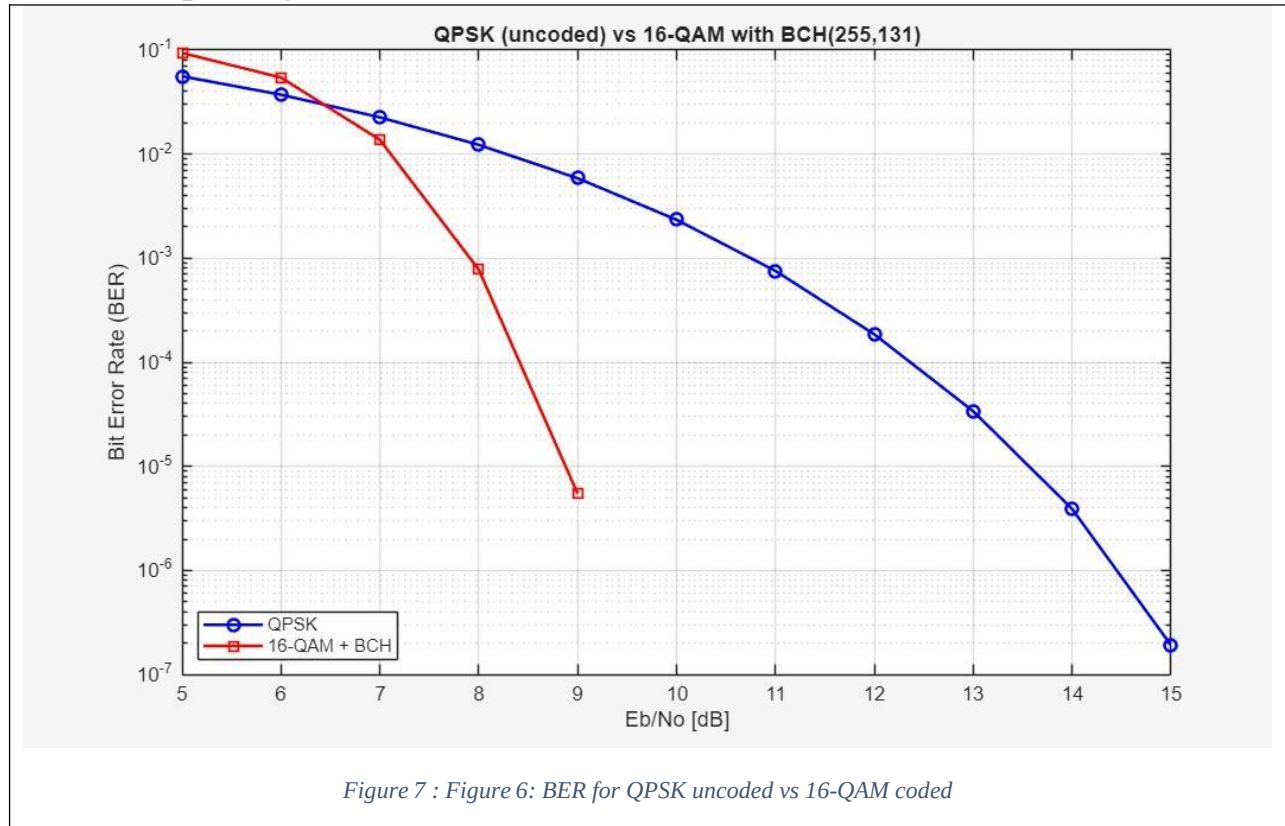
%% ===== PLOT =====
figure;
semilogy(EbNo_dB, ber_qpsk, 'b-o', 'LineWidth',1.5); hold
on;
semilogy(EbNo_dB, ber_16qam, 'r-s', 'LineWidth',1.5);
grid on;
xlabel('Eb/No [dB]');
ylabel('Bit Error Rate (BER)');
title('QPSK (uncoded) vs 16-QAM with BCH(255,131)');
legend('QPSK', '16-QAM + BCH', 'Location', 'southwest');

%% ===== 16-QAM Modulation Function
=====
function [rxsig]=mod16(txbits)
psk16mod=[1+j*1 3+j*1 1+j*3 3+j*3 1-j*1 3-j*1 1-j*3 3j*3
...
-1+j*1 -3+j*1 -1+j*3 -3+j*3 -1-j*1 -3-j*1 -1-j*3 -3-j*3];
m=4;
sigqam16=reshape(txbits,m,length(txbits)/m);
rxsig=(psk16mod(bi2de(sigqam16')+1)); end

%% ===== 16-QAM Demodulation Function
=====
function [rxbits]=demod16(rxsig)
m=4;
psk16demod=[15 14 6 7 13 12 4 5 9 8 0 1 11 10 2 3];
rxsig(real(rxsig)>3)=3+1j*imag(rxsig(real(rxsig)>3));
rxsig(imag(rxsig)>3)=real(rxsig(imag(rxsig)>3))+1j*3;
rxsig(real(rxsig)<-3)=-3+1j*imag(rxsig(real(rxsig)<-3));
rxsig(imag(rxsig)<-3)=real(rxsig(imag(rxsig)<-3))-1j*3;
rxdemod=round(real((rxsig+3+1j*3)/2))+1j*round(imag((rxsig+
3+1j*3)/2));
rxdebi=real(rxdemod)+4*imag(rxdemod);
sigbits=de2bi(psk16demod(rxdebi+1));
rxbits=reshape(sigbits',1,length(sigbits)*m);
end

```

❖ Output Figure



❖ Comments on Figure

This figure shows the BER performance comparison between uncoded QPSK and 16-QAM with BCH(255,131) coding over an AWGN channel. As expected, the coded 16-QAM (red curve) achieves significantly lower BER at moderate E_b/N_0 values due to the error-correcting capability of the BCH code, while the uncoded QPSK (blue curve) continues to decrease gradually with increasing E_b/N_0 .

Question 9:

Write a Matlab code to convolutionally encode 1000 bits by the (2,3,K) code described by:

$g_{1(1)}=[0\ 1]$ $g_{2(1)}=[1\ 1]$ $g_{1(2)}=[1\ 1]$ $g_{2(2)}=[1\ 0]$ $g_{1(3)}=[0\ 0]$ $g_{2(3)}=[1\ 1]$

❖ Matlab code

```
clc;
clear;
close
all;
%% ===== Parameters =====
N = 40; % Change to 500 for 1000 bits (since we process 2
bits at a time)
data = randi([0 1], 1, 2*N); % Generate 2*N bits %%
===== Shift Registers (1 memory per branch)
=====
SR = zeros(3,1);
%% ===== Generator Polynomials
=====
g1 = [0 1;
1 1; 0
0]; g2 =
[1 1;
1 0;
1 1];
%% ===== Outputs storage
===== out1 = []; out2 = []; out3 = [];
combined = []; inputPairs = {}; encodedOutput = {};

%% ===== Convolutional Encoding
=====
for k = 1:N
% Take 2 bits at a time
u1 = data(2*k-1);
```

```

u2 = data(2*k); %
Process first bit
SR = [u1;
SR(1:2)];
y11_1 = mod(g1(1,1)*u1 + g1(1,2)*SR(1),2);
y12_1 = mod(g2(1,1)*u1 + g2(1,2)*SR(1),2);
y21_1 = mod(g1(2,1)*u1 + g1(2,2)*SR(2),2);
y22_1 = mod(g2(2,1)*u1 + g2(2,2)*SR(2),2);
y31_1 = mod(g1(3,1)*u1 + g1(3,2)*SR(3),2);
y32_1 = mod(g2(3,1)*u1 + g2(3,2)*SR(3),2);
% Process second bit
SR = [u2; SR(1:2)];
y11_2 = mod(g1(1,1)*u2 + g1(1,2)*SR(1),2);
y12_2 = mod(g2(1,1)*u2 + g2(1,2)*SR(1),2);
y21_2 = mod(g1(2,1)*u2 + g1(2,2)*SR(2),2);
y22_2 = mod(g2(2,1)*u2 + g2(2,2)*SR(2),2);
y31_2 = mod(g1(3,1)*u2 + g1(3,2)*SR(3),2);
y32_2 = mod(g2(3,1)*u2 + g2(3,2)*SR(3),2);
% Store branch outputs
out1 = [out1 y11_1 y12_1 y11_2 y12_2];
out2 = [out2 y21_1 y22_1 y21_2 y22_2];
out3 = [out3 y31_1 y32_1 y31_2 y32_2];
% Combined output: 3 bits (one from each branch,
using first output) output_bit1 = y11_1; output_bit2
= y21_1; output_bit3 = y31_1;
combined = [combined output_bit1 output_bit2
output_bit3]; inputPairs{k} = sprintf('%d%d', u1, u2);
encodedOutput{k} = sprintf('%d%d%d', output_bit1,
output_bit2, output_bit3); end

%% ===== CREATE TABLE IN FIGURE
=====

fig = uifigure('Name', 'Convolutional Encoder Output',
'Position', [100 100 400 600]);

T = table(inputPairs', encodedOutput', 'VariableNames',
{'inputPairs', 'encodedOutput'});

uit = uitable(fig, 'Data', T, 'Position', [20 20 360 560]);
uit.ColumnName = {'inputPairs', 'encodedOutput'};

```

```

%% ===== DISPLAY IN COMMAND WINDOW
=====
fprintf('\n');
fprintf('%-20s %-20s\n', 'inputPairs', 'encodedOutput');
fprintf('%-20s %-20s\n', '-----', '-----');

for k = 1:N
fprintf('%-20s %-20s\n', inputPairs{k},
encodedOutput{k}); end fprintf('\n');

%% ===== DISPLAY FULL OUTPUT
=====
fprintf('\n===== FULL OUTPUT =====\n');
fprintf('\n%d input bits processed as %d pairs:\n', 2*N,
N);
fprintf('Input_data = ');
fprintf('%d ', data);
fprintf('\n');

fprintf('\nBranch Outputs:\n');
fprintf('Output1 = ');
fprintf('%d ', out1);
fprintf('\n');

fprintf('Output2 = ');
fprintf('%d ', out2);
fprintf('\n');

fprintf('Output3 = ');
fprintf('%d ', out3);
fprintf('\n');

fprintf('\nCombined Output (all %d
bits):\n', length(combined));
fprintf('Output = '); fprintf('%d ',
combined); fprintf('\n\n');

```

❖ Output Figure

inputPairs	encodedOutput
"11"	"110"
"11"	"100"
"11"	"100"
"10"	"100"
"00"	"000"
"01"	"000"
"01"	"010"
"00"	"010"
"01"	"000"
"01"	"010"
"10"	"100"
"11"	"110"
"10"	"100"
"11"	"110"
"10"	"100"
"01"	"000"
"10"	"100"
"11"	"110"
"11"	"100"
"10"	"100"
"00"	"000"
"01"	"000"
"11"	"100"

Figure 8 : Convolution Encoding

[illegible]

Figure 9 : Output bit stream

❖ Comments on Figure

The program implements a convolutional encoder that takes two input bits and produces three encoded bits per pair. Each branch uses a different generator polynomial to encode the input independently. The varying output patterns confirm that the encoder operates correctly.

Appendix: Full Matlab Code

```
clc; clear; close all;

fprintf('===== SELECT PART
=====\\n');
fprintf('Press 3 → Run Part 3 (Uncoded BPSK)\\n');
fprintf('Press 4 → Run Part 4 (Repetition-3 Hard
Decision)\\n');
fprintf('Press 5 → Run Part 5 (Repetition-3 Soft
Decision)\\n');
fprintf('Press 6 → Run Part 6 (BPSK + Hamming (7,4))\\n');
fprintf('Press 7 → Run Part 7 (QPSK + Hamming (15,11))\\n');
fprintf('Press 8 → Run Part 8 (16-QAM + BCH)\\n');
fprintf('Press 9 → Run Part 9 (Convolutional Encoder)\\n');
fprintf('=====\\n');
choice = input('Enter part number:

');

switch
choice case
3 part3();
case 4
part4();
case 5
part5();
case 6
part6();
case 7
part7();
case 8
part8();
case 9
part9();
otherwise
fprintf('✗ Invalid choice. Enter a number from 3 to 9.\\n');
end

%% part 3: (Uncoded
BPSK) function part3()
clear; clc; close all;
```

```

% Number of bits
N = 110000;

% Generate random bits (0/1)
bits = randi([0 1], N, 1);

% BPSK mapping: 0 -> -A , 1 -> +A
A = 1; % amplitude Eb = A^2; % energy
per bit symbols = 2*bits - 1; % gives
-1 and +1

% Eb/No range EbNo_dB = -
3:1:10; ber =
zeros(size(EbNo_dB));

for i = 1:length(EbNo_dB) %
Convert Eb/No from dB to linear
EbNo = 10^(EbNo_dB(i)/10);
% Noise variance per dimension (given in question)
sigma = sqrt((Eb/2) / EbNo);
% AWGN noise
noise = sigma * randn(N,1);
% Received signal r =
symbols + noise; %
Detection (threshold at 0)
detected_bits = r > 0; %
Compute BER
ber(i) = sum(bits ~= detected_bits) / N;
end

% Theoretical BER for BPSK EbNo_linear =
10.^(EbNo_dB/10); ber_theoretical =
qfunc(sqrt(2*EbNo_linear));

% Plot results
figure;
semilogy(EbNo_dB, ber, 'bo-', 'LineWidth', 1.5); hold on;
semilogy(EbNo_dB, ber_theoretical, 'r--', 'LineWidth',
1.5); grid on; xlabel('Eb/No
(dB)'); ylabel('Bit Error Rate
(BER)'); title('BER

```

```

Performance of Uncoded BPSK');
legend('Simulated
BER','Theoretical BER'); end

%% part 4: (Repetition-3 Hard
Decision) function part4() clear; clc;

%% Parameters
Rep = 3; % Repetition factor
Number_of_Information_Bits = 110000;
Information_bits = randi([0,1], 1,
Number_of_Information_Bits);
EbNo_dB = -4:1:10; % Eb/No range
N_EbNo = length(EbNo_dB);

%% --- BPSK with No Code
A = 1;
Eb = A^2;
Mapped_Data = (2*Information_bits - 1) * A;
BER_No_Code = zeros(1, N_EbNo);

for idx = 1:N_EbNo
ebno = EbNo_dB(idx);
Sigma = sqrt((Eb/2) / 10^(ebno/10));
Noise = randn(1, Number_of_Information_Bits);
Rx = Mapped_Data + Sigma*Noise;
DeMapped_Signal = Rx > 0; % 0/1 decision
BER_No_Code(idx) = sum(DeMapped_Signal ~=
Information_bits)/Number_of_Information_Bits;
end

%% --- Repetition-3: Same Energy per Transmitted Bit
Information_bits_rep = repelem(Information_bits, Rep);
A = 1; % same energy per transmitted bit
Eb = A^2;
Mapped_Data_rep = (2*Information_bits_rep - 1) * A;

```

```

BER_HARD_1 = zeros(1, N_EbNo);

for idx = 1:N_EbNo

    ebno = EbNo_dB(idx);
    Sigma = sqrt((Eb/2)/10^(ebno/10));
    Noise = randn(1, length(Information_bits_rep));
    Rx = Mapped_Data_rep + Sigma*Noise;
    DeMapped_Signal = Rx > 0; % 0/1 decision
    Restored_Signal = zeros(1,
    Number_of_Information_Bits); for k =
    1:Number_of_Information_Bits group =
    DeMapped_Signal((k-1)*Rep + 1 : k*Rep);
    Restored_Signal(k) = sum(group) >= 2; % majority vote
end
BER_HARD_1(idx) = sum(Restored_Signal ~=
Information_bits)/Number_of_Information_Bits;
end

%% --- Repetition-3: Same Energy per Information Bit A =
1/sqrt(Rep); % scale amplitude to keep energy per info
bit same
Eb = 1;
Mapped_Data_rep = (2*Information_bits_rep - 1) * A;
BER_HARD_2 = zeros(1, N_EbNo);

for idx = 1:N_EbNo
    ebno = EbNo_dB(idx);
    Sigma = sqrt((Eb/2)/10^(ebno/10));
    Noise = randn(1, length(Information_bits_rep));
    Rx = Mapped_Data_rep + Sigma*Noise;
    DeMapped_Signal = Rx > 0; % 0/1 decision
    Restored_Signal = zeros(1,
    Number_of_Information_Bits); for k =
    1:Number_of_Information_Bits group =
    DeMapped_Signal((k-1)*Rep + 1 : k*Rep);
    Restored_Signal(k) = sum(group) >= 2; % majority vote
end
BER_HARD_2(idx) = sum(Restored_Signal ~=
Information_bits)/Number_of_Information_Bits;
end

```



```

%% --- Plot BER
figure;
semilogy(EbNo_dB, BER_No_Code, 'r', 'LineWidth', 1); hold
on;
semilogy(EbNo_dB, BER_HARD_1, 'b', 'LineWidth', 1);
semilogy(EbNo_dB, BER_HARD_2, 'g', 'LineWidth', 1); grid
on; axis tight; xlabel('Eb/No (dB)'); ylabel('BER');
title('BER: Uncoded BPSK vs Repetition-3 Coding');
legend('Uncoded BPSK', ...
'Repet-3 (Same E per Transmitted Bit)', ...
'Repet-3 (Same E per Information Bit)', ...
'Location', 'southwest');

end

```

```

%% part 5: (Repetition-3 Soft Decision)
function part5()
clc; clear all; close all; %#ok < CLALL >

%% Parameters
Rep = 3; % Repetition factor
Number_of_Information_Bits = 110000;
Information_bits = randi([0,1], 1,
Number_of_Information_Bits);
EbNo_dB = -4:1:10; % Eb/No range
N_EbNo = length(EbNo_dB);
%% --- BPSK with No Code
A = 1;
Eb = A^2;
Mapped_Data = (2*Information_bits - 1) *
A; BER_No_Code = zeros(1, N_EbNo); for idx
= 1:N_EbNo ebno = EbNo_dB(idx);
Sigma = sqrt((Eb/2) / 10^(ebno/10));
Noise = randn(1, Number_of_Information_Bits);
Rx = Mapped_Data + Sigma*Noise;
DeMapped_Signal = Rx > 0; % 0/1 decision

```

```

BER_No_Code(idx) = sum(DeMapped_Signal ~=
Information_bits)/Number_of_Information_Bits;
end
%% --- Repetition-3: Same Energy per Transmitted Bit
Information_bits_rep = repelem(Information_bits, Rep);
A = 1; % same energy per transmitted bit
Eb = A^2;
Mapped_Data_rep = (2*Information_bits_rep - 1) * A;
BER_SOFT_1 = zeros(1,
N_EbNo); for idx = 1:N_EbNo
ebno = EbNo_dB(idx);
Sigma = sqrt((Eb/2)/10^(ebno/10));
Noise = randn(1, length(Information_bits_rep));
Rx = Mapped_Data_rep + Sigma*Noise;
groups_a = reshape(Rx, Rep, []);

sum_soft_a = sum(groups_a, 1);
det_soft_a = sum_soft_a > 0;

BER_SOFT_1(idx) = sum(det_soft_a ~=
Information_bits)/Number_of_Information_Bits;
end
%% --- Repetition-3: Same Energy per Information Bit A =
1/sqrt(Rep); % scale amplitude to keep energy per info
bit same
Eb = 1;
Mapped_Data_rep = (2*Information_bits_rep - 1) * A;
BER_SOFT_2 = zeros(1,
N_EbNo); for idx = 1:N_EbNo
ebno = EbNo_dB(idx);
Sigma = sqrt((Eb/2)/10^(ebno/10));
Noise = randn(1, length(Information_bits_rep));
Rx = Mapped_Data_rep + Sigma*Noise;
groups_b = reshape(Rx, Rep, []);

sum_soft_b = sum(groups_b, 1);
det_soft_b = sum_soft_b > 0;

BER_SOFT_2(idx) = sum(det_soft_b ~=

```

```

Information_bits)/Number_of_Information_Bits;
end
%% --- Plot BER
figure;
semilogy(EbNo_dB, BER_No_Code, 'r', 'LineWidth', 2); hold
on;
semilogy(EbNo_dB, BER_SOFT_1, 'b', 'LineWidth',
2); semilogy(EbNo_dB, BER_SOFT_2, 'g',
'LineWidth', 2); grid on; axis tight;
xlabel('Eb/No (dB)'); ylabel('BER');
title('BER: Uncoded BPSK vs Repetition-3 Coding
(Soft decision decoding)'); legend('Uncoded BPSK',
...
'Repet-3 (Same E per Transmitted Bit)', ...
'Repet-3 (Same E per Information Bit)', ...
'Location', 'southwest');

end

```

```

%% part 6: (BPSK +
Hamming) function part6()
clc; clear; close all;

%% ===== Parameters =====
bitsNumber = 110000; % Number of information bits
Eb = 1; % Energy per information bit A =
1; % BPSK signal amplitude ebno_dB = -
2:1:10; % Eb/N0 range in dB n = 7; %
Hamming codeword length (7 bits) k = 4;
% Hamming message length (4 bits) rate =
k/n; % Code rate of Hamming (7,4)

%% ===== Generate random bits
=====
Bits = randi([0 1], 1, bitsNumber); % Generate random
binary information bits

```

```

%% ===== BPSK Without Coding
=====
BPSKSymbols = A * (2*Bits - 1); % Map 0->-1, 1->+1 for
BPSK
BER_uncoded = zeros(1,length(ebno_dB));

for idx = 1:length(ebno_dB)
EbN0 = ebno_dB(idx);
sigma = sqrt((Eb/2)/10^(EbN0/10)); % Noise standard
deviation for AWGN
Noise = sigma * randn(1,bitsNumber); % Generate AWGN noise
Received = BPSKSymbols + Noise; % Received signal after
noise
ReceivedBits = Received > 0; % Hard decision
ErrorBits = sum(Bits ~= double(ReceivedBits)); % Count bit
errors
BER_uncoded(idx) = ErrorBits / bitsNumber;
end

%% ===== BPSK with Hamming (7,4) Same
Energy per Transmitted Bit =====
HammCodeBits = encode(Bits, n, k, 'hamming/binary'); %
Encode using Hamming (7,4)
A = 1; % Keep amplitude same per transmitted bit
BPSKSymbols_case_a = A * (2*HammCodeBits - 1); % BPSK
mapping for coded bits
BER_case_a = zeros(1,length(ebno_dB));

for idx = 1:length(ebno_dB)
EbN0 = ebno_dB(idx);
sigma = sqrt((Eb/2)/10^(EbN0/10));
Noise = sigma * randn(1,length(HammCodeBits)); % Generate
noise
Received = BPSKSymbols_case_a + Noise; % Received signal
ReceivedBits = Received > 0; % Hard decision
DecodedBits = decode(ReceivedBits, n, k,
'hamming/binary'); % Decode
ErrorBits = sum(Bits ~=
double(DecodedBits)); BER_case_a(idx) =
ErrorBits / bitsNumber; end

```

```

%% ===== BPSK with Hamming (7,4) Same
Energy per Information Bit =====
A = sqrt(rate); % Scale amplitude to keep same energy per
info bit
BPSKSymbols_case_b = A * (2*HammCodeBits - 1); % BPSK
mapping
BER_case_b = zeros(1,length(ebno_dB)); % Preallocate BER
array

for idx = 1:length(ebno_dB)
EbN0 = ebno_dB(idx);
sigma = sqrt((Eb/2)/10^(EbN0/10));
Noise = sigma * randn(1,length(HammCodeBits));
Received = BPSKSymbols_case_b + Noise; % Received signal
ReceivedBits = Received > 0; % Hard decision
DecodedBits = decode(ReceivedBits, n, k,
'hamming/binary'); % Decode
ErrorBits = sum(Bits ~=
double(DecodedBits)); BER_case_b(idx) =
ErrorBits / bitsNumber; end

%% ===== Plot BER Comparison
=====
figure;
semilogy(ebno_dB, BER_uncoded,'k-','LineWidth',2); hold
on; semilogy(ebno_dB, BER_case_a,'r-','LineWidth',2);
semilogy(ebno_dB, BER_case_b,'b-','LineWidth',2); grid on;
xlabel('E_b/N_0 (dB)');
ylabel('BER');
title('BER Comparison of BPSK uncoded and with Hamming
(7,4) Coding');
legend('BPSK Uncoded',...
'BPSK Hamming (Same Energy per Transmission
Bit)',... 'BPSK Hamming (Same Energy per Info
Bit)'); end

```

```

%% part 7: (QPSK +
Hamming) function part7()
clc; clear; close all;

%% ===== Parameters =====
bitsNumber = 110000; % Number of information bits
Eb = 1; % Energy per information bit A =
1; % BPSK signal amplitude ebno_dB = -
2:1:10; % Eb/N0 range in dB n = 7; %
Hamming codeword length (7 bits) k = 4;
% Hamming message length (4 bits) rate =
k/n; % Code rate of Hamming (7,4)

%% ===== Generate random bits
=====
Bits = randi([0 1], 1, bitsNumber); % Generate random
binary information bits

%% ===== BPSK Without Coding
=====
BPSKSymbols = A * (2*Bits - 1); % Map 0->-1, 1->+1 for
BPSK
BER_uncoded = zeros(1,length(ebno_dB));

for idx = 1:length(ebno_dB)
EbN0 = ebno_dB(idx);
sigma = sqrt((Eb/2)/10^(EbN0/10)); % Noise standard
deviation for AWGN
Noise = sigma * randn(1,bitsNumber); % Generate AWGN noise
Received = BPSKSymbols + Noise; % Received signal after
noise
ReceivedBits = Received > 0; % Hard decision
ErrorBits = sum(Bits ~= double(ReceivedBits)); % Count bit
errors
BER_uncoded(idx) = ErrorBits / bitsNumber;
end

%% ===== BPSK with Hamming (7,4) Same
Energy per Transmitted Bit =====
HammCodeBits = encode(Bits, n, k, 'hamming/binary'); %
Encode using Hamming (7,4)

```

```

A = 1; % Keep amplitude same per transmitted bit
BPSKSymbols_case_a = A * (2*HammCodeBits - 1); % BPSK
mapping for coded bits
BER_case_a = zeros(1,length(ebno_dB));

for idx = 1:length(ebno_dB)
EbN0 = ebno_dB(idx);
sigma = sqrt((Eb/2)/10^(EbN0/10));
Noise = sigma * randn(1,length(HammCodeBits)); % Generate
noise
Received = BPSKSymbols_case_a + Noise; % Received signal
ReceivedBits = Received > 0; % Hard decision
DecodedBits = decode(ReceivedBits, n, k,
'hamming/binary'); % Decode
ErrorBits = sum(Bits ~=
double(DecodedBits)); BER_case_a(idx) =
ErrorBits / bitsNumber; end
%% ===== BPSK with Hamming (7,4) Same
Energy per Information Bit =====
A = sqrt(rate); % Scale amplitude to keep same energy per
info bit
BPSKSymbols_case_b = A * (2*HammCodeBits - 1); % BPSK
mapping
BER_case_b = zeros(1,length(ebno_dB)); % Preallocate BER
array

for idx = 1:length(ebno_dB)
EbN0 = ebno_dB(idx);
sigma = sqrt((Eb/2)/10^(EbN0/10));
Noise = sigma * randn(1,length(HammCodeBits));
Received = BPSKSymbols_case_b + Noise; % Received signal
ReceivedBits = Received > 0; % Hard decision
DecodedBits = decode(ReceivedBits, n, k,
'hamming/binary'); % Decode
ErrorBits = sum(Bits ~=
double(DecodedBits)); BER_case_b(idx) =
ErrorBits / bitsNumber; end

%% ===== Plot BER Comparison

```

```

=====
figure;
semilogy(ebno_dB, BER_uncoded,'k-','LineWidth',2); hold
on; semilogy(ebno_dB, BER_case_a,'r-','LineWidth',2);
semilogy(ebno_dB, BER_case_b,'b-','LineWidth',2); grid on;
xlabel('E_b/N_0 (dB)');
ylabel('BER');
title('BER Comparison of BPSK uncoded and with Hamming
(7,4) Coding');
legend('BPSK Uncoded',...
'BPSK Hamming (Same Energy per Transmission
Bit)',... 'BPSK Hamming (Same Energy per Info
Bit)'); end

%% part 8: (QPSK + Hamming)
function part8()
%% ===== Clear workspace =====
clear; close all; clc;

%% ===== PARAMETERS =====
Eb = 2.5;
EbNo_dB = 5:15; % Eb/No range in dB
Nbits_total = 26200000;

% BCH code parameters
(255,131) n_bch = 255; k_bch =
131;

%% ===== BCH Encoder/Decoder
===== bchEnc = comm.BCHEncoder(n_bch, k_bch);
bchDec = comm.BCHDecoder(n_bch, k_bch);

%% ===== BER Arrays
===== ber_qpsk =
zeros(size(EbNo_dB)); ber_16qam =
zeros(size(EbNo_dB));

%% ===== QPSK SIMULATION =====

```



```

for idx = 1:length(EbNo_dB)
EbNo =
10^(EbNo_dB(idx)/10); %
Adaptive number of bits if
EbNo_dB(idx) <= 8 Nbits =
2620000; else
Nbits = Nbits_total;
end
% Generate random bits txBits =
randi([0 1], 1, Nbits);
% QPSK mapping: 00->1+j, 01->-1+j, 11->-1-j, 10->1-j
txSym = 1-2*txBits(1:2:end) + 1j*(1-2*txBits(2:2:end));
% Add AWGN
N0 = 2*Eb/(2*EbNo); % QPSK has 2 bits/symbol
noise =
sqrt(N0/2)*(randn(size(txSym))+1j*randn(size(txSym)));
rxSym = txSym + noise;
% QPSK demodulation
rxBits =
zeros(1,Nbits);
rxBits(1:2:end) =
real(rxSym)<0;
rxBits(2:2:end) =
imag(rxSym)<0;
% BER computation
ber_qpsk(idx) = sum(rxBits~=txBits)/Nbits;
end

%% ===== 16-QAM + BCH SIMULATION
===== for idx =
1:length(EbNo_dB) EbNo =
10^(EbNo_dB(idx)/10); %
Adaptive number of bits if
EbNo_dB(idx) <= 8 Nbits =
2620000; else
Nbits = Nbits_total;
end
% Generate random bits txBits =
randi([0 1], 1, Nbits); % BCH encoding
block by block nBlocks =
ceil(length(txBits)/k_bch); encBits =

```

```

zeros(1, nBlocks*n_bch); for blk =
1:nBlocks startIdx = (blk-1)*k_bch + 1;
endIdx = min(blk*k_bch, length(txBits));
dataBlock = txBits(startIdx:endIdx); %
Pad last block if needed if
length(dataBlock) < k_bch
dataBlock = [dataBlock zeros(1,k_bch-length(dataBlock))];
end
% Convert to column vector
dataBlockCol = dataBlock(:);
% Encode
encBlockCol = step(bchEnc, dataBlockCol);
% Store in row vector
encBits((blk-1)*n_bch + 1 : blk*n_bch) = encBlockCol(:)';
end
% 16-QAM modulation using provided function
txSym = modl6(encBits); % Add AWGN
m = 4; % 16-QAM: 4 bits/symbol
N0 = 4*Eb/(2*EbNo); % 16-QAM has 4 bits/symbol
noise =
sqrt(N0/2)*(randn(size(txSym))+1j*randn(size(txSym)));
rxSym = txSym + noise;
% 16-QAM demodulation using provided function
rxBits = demodl6(rxSym); % BCH decoding block
by block decBits = zeros(1,length(txBits));
for blk = 1:nBlocks startIdx = (blk-1)*n_bch
+ 1; endIdx = blk*n_bch;
rxBlock = rxBits(startIdx:endIdx);
% Convert to column vector
rxBlockCol = rxBlock(:);
% Decode
decBlockCol = step(bchDec, rxBlockCol);
% Trim padding for last block
if blk==nBlocks && length(decBlockCol) > length(txBits) -
(nBlocks-1)*k_bch
decBlockCol = decBlockCol(1:length(txBits) - (nBlocks-
1)*k_bch);
end
decBits((blk-1)*k_bch + 1 : min(blk*k_bch,length(txBits)))

```

```

= decBlockCol(:)';
end
% BER computation
ber_16qam(idx) = sum(decBits~=txBits)/length(txBits);
end

%% ===== PLOT =====
figure;
semilogy(EbNo_dB, ber_qpsk, 'b-o','LineWidth',1.5); hold
on;
semilogy(EbNo_dB, ber_16qam, 'r-
s','LineWidth',1.5); grid on; xlabel('Eb/No [dB]');
ylabel('Bit Error Rate (BER)');
title('QPSK (uncoded) vs 16-QAM with BCH(255,131)');
legend('QPSK','16-QAM + BCH','Location','southwest');

%% ===== 16-QAM Modulation Function
===== function
[rxsig]=mod16(txbits)
psk16mod=[1+j*1 3+j*1 1+j*3 3+j*3 1-j*1 3-j*1 1-j*3 3j*3
...
-1+j*1 -3+j*1 -1+j*3 -3+j*3 -1-j*1 -3-j*1 -1-j*3 -3-j*3];
m=4;

```

```
sigqam16=reshape(txbits,m,length(txbits)/m);
rxsig=(psk16mod(bi2de(sigqam16')+1)); end

%% ===== 16-QAM Demodulation Function
=====
function [rxbits]=demod16(rxsig)
m=4;
psk16demod=[15 14 6 7 13 12 4 5 9 8 0 1 11 10 2 3];
rxsig(real(rxsig)>3)=3+1j*imag(rxsig(real(rxsig)>3));
rxsig(imag(rxsig)>3)=real(rxsig(imag(rxsig)>3))+1j*3;
rxsig(real(rxsig)<-3)=-3+1j*imag(rxsig(real(rxsig)<-3));
rxsig(imag(rxsig)<-3)=real(rxsig(imag(rxsig)<-3))-1j*3;
rxdemod=round(real((rxsig+3+1j*3)/2))+1j*round(imag((rxsig+
3+1j*3)/2));
```

```

rxdebi=real(rxdemod)+4*imag(rxdemod);
sigbits=de2bi(psk16demod(rxdebi+1));
rxbits=reshape(sigbits',1,length(sigbits)*m);
end

```

```

end

```

```

%% part 9: (Convolutional Encoder)

```

```

function part9()

```

```

clc;

```

```

clear;

```

```

close

```

```

all;

```

```

%% ===== Parameters =====

```

```

N = 40; % Change to 500 for 1000 bits (since we process 2
bits at a time)

```

```

data = randi([0 1], 1, 2*N); % Generate 2*N bits %%

```

```

===== Shift Registers (1 memory per branch)

```

```

=====

```

```

SR = zeros(3,1);

```

```

%% ===== Generator Polynomials

```

```

=====

```

```

g1 = [0

```

```

1; 1 1; 0

```

```

0]; g2 =

```

```

[1 1; 1

```

```

0;

```

```

1 1];

```

```

%% ===== Outputs storage

```

```

===== out1 = []; out2 = []; out3 = [];

```

```

combined = []; inputPairs = {}; encodedOutput = {};

```

```

%% ===== Convolutional Encoding

```

```

=====
for k = 1:N % Take 2
bits at a time u1 =
data(2*k-1); u2 =
data(2*k); % Process
first bit SR = [u1;
SR(1:2)];
y11_1 = mod(g1(1,1)*u1 + g1(1,2)*SR(1),2);
y12_1 = mod(g2(1,1)*u1 + g2(1,2)*SR(1),2);
y21_1 = mod(g1(2,1)*u1 + g1(2,2)*SR(2),2);
y22_1 = mod(g2(2,1)*u1 + g2(2,2)*SR(2),2);
y31_1 = mod(g1(3,1)*u1 + g1(3,2)*SR(3),2);
y32_1 = mod(g2(3,1)*u1 + g2(3,2)*SR(3),2);
% Process second bit
SR = [u2; SR(1:2)];
y11_2 = mod(g1(1,1)*u2 + g1(1,2)*SR(1),2);
y12_2 = mod(g2(1,1)*u2 + g2(1,2)*SR(1),2);
y21_2 = mod(g1(2,1)*u2 + g1(2,2)*SR(2),2);
y22_2 = mod(g2(2,1)*u2 + g2(2,2)*SR(2),2);
y31_2 = mod(g1(3,1)*u2 + g1(3,2)*SR(3),2);
y32_2 = mod(g2(3,1)*u2 + g2(3,2)*SR(3),2);
% Store branch outputs out1 = [out1
y11_1 y12_1 y11_2 y12_2]; out2 =
[out2 y21_1 y22_1 y21_2 y22_2]; out3
= [out3 y31_1 y32_1 y31_2 y32_2];
% Combined output: 3 bits (one from each branch, using
first output) output_bit1 = y11_1; output_bit2 = y21_1;
output_bit3 = y31_1;
combined = [combined output_bit1 output_bit2
output_bit3]; inputPairs{k} = sprintf("%d%d", u1, u2);
encodedOutput{k} = sprintf("%d%d%d", output_bit1,
output_bit2, output_bit3); end

%% ===== CREATE TABLE IN FIGURE
=====
fig = uifigure('Name', 'Convolutional Encoder Output',
'Position', [100 100 400 600]);

T = table(inputPairs', encodedOutput', 'VariableNames',
{'inputPairs', 'encodedOutput'});

```

```

uit = uitable(fig, 'Data', T, 'Position', [20 20 360
560]); uit.ColumnName = {'inputPairs', 'encodedOutput'};

%% ===== DISPLAY IN COMMAND WINDOW
=====
fprintf('\n');
fprintf('%-20s %-20s\n', 'inputPairs', 'encodedOutput');
fprintf('%-20s %-20s\n', '-----', '-----');

for k = 1:N
fprintf('%-20s %-20s\n', inputPairs{k},
encodedOutput{k}); end fprintf('\n');

%% ===== DISPLAY FULL OUTPUT
=====
fprintf('\n===== FULL OUTPUT =====\n');
fprintf('\n%d input bits processed as %d pairs:\n', 2*N,
N);
fprintf('Input_data = ');
fprintf('%d ', data);
fprintf('\n');
fprintf('\nBranch
Outputs:\n');

```

```
fprintf('Output1 = ');  
fprintf('%d ', out1);  
fprintf('\n');  
  
fprintf('Output2 = ');  
fprintf('%d ', out2);  
fprintf('\n');  
  
fprintf('Output3 = ');  
fprintf('%d ', out3);  
fprintf('\n');  
  
fprintf('\nCombined Output (all %d  
bits):\n', length(combined));  
fprintf('Output = '); fprintf('%d ',  
combined); fprintf('\n\n');  
  
end
```